18

Iterative Methods

Historical Note

Gauss described an iterative method in a letter to Gerling in December 1823 and recommended it in terms no modern paper would risk: "I recommend this modus operandi. You will hardly eliminate directly any more, at least not when you have more than two unknowns. The indirect procedure can be done while half asleep, or while thinking about other things." The letter went unpublished for decades and the method reappeared under Seidel's name in 1874; Jacobi's variant dates from 1845. The decisive idea came much later. Hestenes and Stiefel, working separately and then together at the National Bureau of Standards, published conjugate gradients in 1952, and the method was promptly filed away as a curiosity: it terminates in n steps, so it looked like a direct method, and as a direct method it compared badly with elimination. Twenty years passed before the sparse matrix community recognised that terminating in n steps is irrelevant when n is a million and what matters is the accuracy after fifty. Saad and Schultz gave the nonsymmetric counterpart, GMRES, in 1986.

Chapter 17 ended by observing that the cubic cost of a factorisation is a wall, and that the way past it is not a better constant but a different kind of algorithm. This chapter describes that kind. The common feature is that the matrix is never factored and often never even formed: the only thing required of A is the ability to compute Ax.

18.1 Why factorisation fails at scale

A sparse matrix with about c nonzero entries per row costs roughly $2cn$ operations per matrix-vector product and cn words to store. Both are linear in n. Its factors are not.

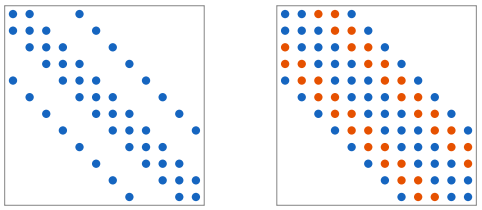


Figure 18.1. Fill-in. On the left, a banded matrix with three diagonals of nonzeros. On the right, the combined pattern of its L and U factors, which occupy the entire band: every position between the outermost diagonals acquires an entry. For this small banded example the growth is modest, from 50 to 88. For a matrix from a three-dimensional grid it is catastrophic, and a factorisation that would fit in memory as A does not fit as LU. Reordering the rows and columns can delay the damage but not prevent it.

Key Idea

An iterative method needs only a procedure that returns Ax for a given x. It never asks for the entries of A, which matters more than it sounds: in a finite difference or finite element calculation, A is defined by a stencil and applying it is a loop over neighbours; in a graph problem, A is the adjacency structure; in a machine learning problem, Ax may be a composition of operations no one would want to write as a matrix. All of these can be solved iteratively and none can be factored.

18.2 Stationary iterations

The oldest idea is to split the matrix into an easy part and the rest.

Definition 18.1 (Splitting and the associated iteration)**.** *Write $A = M - N$ with M invertible and cheap to solve with. The associated iteration is*

$$Mx_{k+1} = Nx_k + b, \qquad \text{that is} \qquad x_{k+1} = Gx_k + M^{-1}b, \quad G = M^{-1}N.$$

Taking $M = D$, the diagonal of A, gives Jacobi*; taking M to be the lower triangular part of A including the diagonal gives* Gauss–Seidel.

Theorem 18.2 (When a stationary iteration converges)**.** *Let x solve $Ax = b$ and let $e_k = x_k - x$. Then $e_k = G^k e_0$. If G is diagonalisable, the iteration converges for every starting vector if and only if*

$$\rho(G) \;=\; \max\big\{|\lambda| : \lambda \in \operatorname{Spec}(G)\big\} \;<\; 1,$$

and then $\|e_k\|$ decreases asymptotically by a factor $\rho(G)$ per step. The quantity $\rho(G)$ is called the spectral radius*.*

Proof. Since $Mx = Nx + b$, subtracting the iteration gives $Me_{k+1} = Ne_k$, so $e_{k+1} = Ge_k$ and $e_k = G^k e_0$ by induction. Now let G have an eigenbasis $v_1, \dots, v_n$ with eigenvalues λ_i and write $e_0 = \sum_i c_i v_i$. By Proposition 9.18(a),

$$e_k = \sum_{i} c_i \lambda_i^k \, v_i .$$

If every $|\lambda_i| < 1$ then every term tends to $\mathbf{0}$. Conversely if some $\left|\lambda_j\right| \geq 1$, starting with $e_0 = v_j$ gives $\|e_k\| = \left|\lambda_j\right|^k \left\|v_j\right\|$, which does not tend to zero. The asymptotic rate is governed by the largest $|\lambda_i|$ present. For non-diagonalisable G the same conclusion holds, with the Jordan form contributing polynomial factors in k that do not affect it. $\square$

Example 18.3 (The one-dimensional Laplacian, and the bad news)**.** *Let A be $n \times n$ tridiagonal with 2 on the diagonal and -1 beside it, the matrix of a discretised second derivative. Its eigenvalues are known exactly:*

$$\lambda_j = 2 - 2\cos\theta_j = 4\sin^2\frac{\theta_j}{2}, \qquad \theta_j = \frac{j\pi}{n+1}, \quad j = 1, \dots, n.$$

For Jacobi, M is twice the identity and $G = \mathbf{1} - A/2$, whose eigenvalues are $\cos\theta_j$. So

$$\rho(G) = \cos\frac{\pi}{n+1} \approx 1 - \frac{\pi^2}{2(n+1)^2},$$

which is smaller than 1, so Jacobi converges, and hopelessly close to 1, so it converges slowly: reducing the error by a fixed factor takes about $2n^2/\pi^2$ iterations. Meanwhile $\kappa(A) = \lambda_n/\lambda_1 \approx 4(n+1)^2/\pi^2$ grows like n^2, and it is no accident that the two n^2 are the same n^2. A method whose rate is governed by κ rather than by $\sqrt{\kappa}$ is what we shall want to improve on.

18.3 Krylov subspaces

Definition 18.4 (Krylov subspace)**.** *For a matrix A and vector v, the kth* Krylov subspace *is*

$$K_k(A, v) = \operatorname{span}\left\{v,\ Av,\ A^2v,\ \ldots,\ A^{k-1}v\right\}.$$

Key Idea

After k matrix-vector products starting from v, the vectors $v, Av, \ldots, A^{k-1}v$ are everything you have, so $K_k(A, v)$ is everything you could possibly form by taking combinations. The best conceivable iterate after k products is therefore the best approximation to the answer drawn from that subspace, and every method in this section is an answer to the question of what "best" should mean. Conjugate gradients minimises one norm of the error; GMRES minimises the residual. Nothing else distinguishes them.

Caution

Do not use $v, Av, A^2v, \ldots$ as a basis. That sequence is the power method of Section 9.6, so the vectors line up along the dominant eigendirection and become numerically dependent within a dozen steps. What is actually built is an orthonormal basis of the same subspace, by Gram–Schmidt applied as each new vector arrives; the resulting recurrence is called Arnoldi in general and Lanczos when A is symmetric. This is Chapter 12 doing the work inside the algorithm, and it is why the Caution there about classical versus modified Gram–Schmidt matters in practice.

Figure 18.2 shows what actually governs the speed. After k steps the error is $p(A)$ applied to the initial error for some polynomial p of degree k with $p(0) = 1$, and the method does as well as the best such polynomial, so a spectrum sitting in a few tight clusters is easy however wide the extremes are spread.

18.4 Conjugate gradients

Throughout this section A is symmetric and positive definite, so that Theorem 14.14 gives $\langle Ax, x\rangle > 0$ whenever x is not the zero vector.

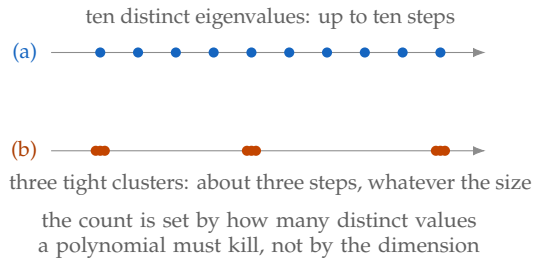


Figure 18.2. What actually governs the speed of a Krylov method. After k steps the error is $p(A)$ applied to the initial error, for some polynomial p of degree k with $p(0) = 1$, and the method does as well as the best such polynomial. If the spectrum has m distinct points, a polynomial of degree m can vanish at all of them, so convergence is exact in m steps however large the matrix is. Case (b) has thirty eigenvalues in three tight clusters and converges in about three steps; case (a) has ten spread out and needs ten. This is also the whole theory of preconditioning: a good preconditioner is one that clusters the spectrum, and the condition number is only a crude summary of how well it has done so.

Proposition 18.5 (Solving is minimising)**.** *Define* $\varphi(x) = \tfrac{1}{2}\langle Ax, x\rangle - \langle b, x\rangle$. *If* $Ax_* = b$ *then for every* x,

$$\varphi(x) - \varphi(x_*) = \tfrac{1}{2}\langle Ae, e\rangle, \qquad e = x - x_*,$$

so φ *has a unique minimum, at the solution.*

Proof. Write $x = x_* + e$ and expand, using symmetry of A and $b = Ax_*$:

$$\tfrac{1}{2}\langle A(x_* + e), x_* + e\rangle = \tfrac{1}{2}\langle Ax_*, x_*\rangle + \langle Ax_*, e\rangle + \tfrac{1}{2}\langle Ae, e\rangle,$$

$$\langle b, x_* + e\rangle = \langle b, x_*\rangle + \langle Ax_*, e\rangle.$$

Subtracting, the cross terms cancel and $\varphi(x) = \varphi(x_*) + \tfrac{1}{2}\langle Ae, e\rangle$. The last term is positive unless $e = \mathbf{0}$. □

The quantity $\|e\|_A = \sqrt{\langle Ae, e\rangle}$ is a norm, since A is positive definite; it is the one conjugate gradients minimises.

Definition 18.6 (Conjugate gradients)**.** *Given* x_0, *set* $r_0 = b - Ax_0$ *and* $p_0 = r_0$, *and for* $k = 0, 1, 2, \ldots$

$$\alpha_k = \frac{\langle r_k, r_k\rangle}{\langle Ap_k, p_k\rangle}, \qquad x_{k+1} = x_k + \alpha_k p_k, \qquad r_{k+1} = r_k - \alpha_k A p_k,$$

$$\beta_k = \frac{\langle r_{k+1}, r_{k+1}\rangle}{\langle r_k, r_k\rangle}, \qquad p_{k+1} = r_{k+1} + \beta_k p_k.$$

Each iteration costs one product Ap_k, two inner products and a few vector updates, and the storage is four vectors regardless of k.

Theorem 18.7 (What conjugate gradients achieves)**.** *The iterate x_k minimises $\|x - x_*\|_A$ over the affine set $x_0 + K_k(A, r_0)$. In exact arithmetic $x_n = x_*$, and for every k,*

$$\|x_k - x_*\|_A \;\le\; 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^{k} \|x_0 - x_*\|_A\,, \qquad \kappa = \kappa(A).$$

The bound is proved in Chapter 44: the argument recasts the minimisation as one over polynomials p with $p(0) = 1$ evaluated at the eigenvalues of A, and then exhibits a good such polynomial built from Chebyshev polynomials. The optimality over K_k and the termination at step n are straightforward once the recurrences are checked, and the termination happens because K_n is the whole space.

Key Idea

The square root is the entire point. For the Laplacian of Example 18.3 we have $\kappa \sim n^2$, so Jacobi needs $O(n^2)$ iterations while conjugate gradients needs $O(\sqrt{\kappa}) = O(n)$. At $n = 10^6$ that is the difference between 10^{12} iterations and 10^6, which is the difference between impossible and routine. Note also what the bound is: an upper bound on the worst case. Actual convergence is often much faster, because it depends on how the eigenvalues are *distributed* and not only on the ratio of the extremes, and this is exactly the loophole preconditioning exploits.

Figure 18.3 plots the bound for two condition numbers. It is geometric, so it is a straight line on a logarithmic scale, and its slope is set by $\sqrt{\kappa}$ rather than κ.

18.5 GMRES, for the rest

When A is not symmetric there is no quadratic to minimise, and the natural substitute is the residual itself.

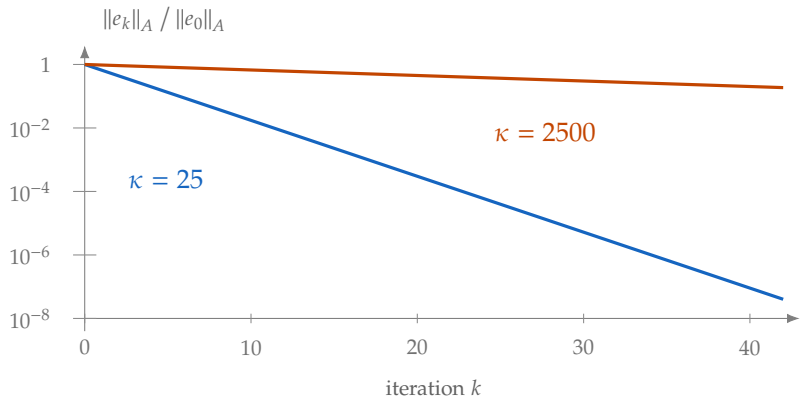


Figure 18.3. The conjugate gradient bound of Theorem 18.7, for two condition numbers. The bound is geometric, so it is a straight line on this logarithmic scale, and its slope is set by $\sqrt{\kappa}$ rather than κ. A hundredfold worse conditioning costs a tenfold slower rate, not a hundredfold one. Real runs usually lie below these lines, sometimes far below, because the bound uses only the two extreme eigenvalues and the method is sensitive to all of them.

Definition 18.8 (GMRES). *The kth GMRES iterate is the $x \in x_0 + K_k(A, r_0)$ minimising $\|b - Ax\|$. In practice Arnoldi supplies an orthonormal basis $q_1, \dots, q_k$ of the Krylov subspace together with a small $(k+1) \times k$ matrix H_k satisfying $AQ_k = Q_{k+1}H_k$, and the minimisation reduces to a least squares problem of that small size, solved by the QR method of Proposition 13.11.*

Key Idea

Chapters 12 and 13 are running inside this algorithm. Gram–Schmidt builds the basis, a small least squares problem is solved at each step, and QR is used because the normal equations would square the condition number. A large sparse problem has been turned into a sequence of tiny dense ones.

Caution

GMRES keeps every basis vector, so its storage and its work per step both grow with k. Practical codes restart after a fixed number of steps, discarding the subspace and beginning again, which loses the optimality and can stall. There is also no convergence theory for nonsymmetric A as clean as Theorem 18.7: the eigenvalue distribution matters, but so does the conditioning of the eigenvector basis, and

a nonsymmetric matrix can have excellent eigenvalues and still be solved slowly. Symmetry is worth a great deal, which is one reason Chapter 14 spent so long on it.

18.6 Preconditioning

Definition 18.9 (Preconditioner)**.** *A* preconditioner *is an invertible M, cheap to solve with, such that the transformed system*

$$M^{-1}Ax = M^{-1}b$$

is easier for the iterative method than the original. For conjugate gradients M must also be symmetric positive definite, and the transformation is applied symmetrically so as to preserve symmetry.

Figure 18.4 shows what is being bought. Before preconditioning the spectrum is spread over four decades, for which the bound of Theorem 18.7 promises little; after it the spectrum is gathered together, and the bound becomes useful.

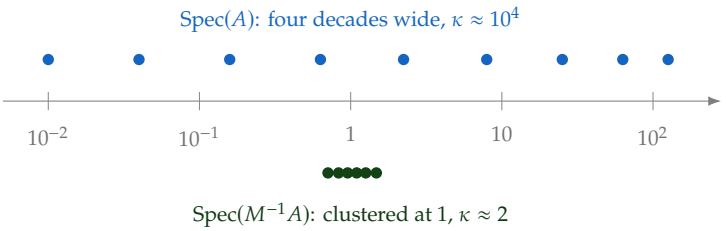


Figure 18.4. What a preconditioner is for. The upper row is a spectrum spread over four decades, for which Theorem 18.7 promises little. The lower row is the spectrum after preconditioning, gathered near 1. Note that the aim is not to make the matrix smaller or sparser; it is to move the eigenvalues, and clustering them anywhere works as well as clustering them at 1.

Example 18.10 (Preconditioners in use)**.** *(i) $M = \operatorname{diag}(A)$, the Jacobi or diagonal preconditioner. Almost free, and it fixes nothing but bad scaling; still worth trying first.*

(ii) Incomplete Cholesky or incomplete LU: run the factorisation of Chapter 15 or of Chapter 4 but discard fill-in below a threshold, so that Figure 18.1 is prevented from

happening. The result is not A but is often close enough.

(iii) Multigrid, for systems arising from elliptic differential equations. It exploits the fact that the slowly converging error components of Example 18.3 are exactly the smooth ones, which are cheap to correct on a coarser grid. For its intended problems it achieves $\kappa(M^{-1}A)$ bounded independently of n, which is as good as the subject gets.

(iv) Domain decomposition, which splits the problem across subregions and is what makes these methods run on many processors.

Key Idea

The Krylov method is generic and the preconditioner is not. Conjugate gradients knows nothing about where A came from; a good preconditioner knows everything, and is built from the physics, the geometry or the graph structure behind the matrix. That division is why the field looks the way it does: the iterative methods themselves were largely settled by the 1980s, and essentially all subsequent work is about preconditioning.

Caution

The preconditioner must be cheap to apply, and there is a trivial trap at each end. Taking $M = \mathbf{1}$ preconditions nothing. Taking $M = A$ gives convergence in a single step and costs exactly as much as solving the original problem, so nothing has been gained. Every useful preconditioner lives between those extremes, and judging the trade-off is a matter of measurement rather than theory: the quantity to minimise is total time, not iteration count, and a preconditioner that halves the iterations while tripling the cost per iteration is a loss.

18.7 What to use when

Key Idea

A summary of the whole of Part VII.

- Small or dense: factor. LU with partial pivoting for a general system, Cholesky (Chapter 15) if symmetric positive definite, QR for least squares.
- Large and sparse, symmetric positive definite: preconditioned conjugate gradients.

- Large and sparse, symmetric but indefinite: MINRES, which minimises the residual over the same Krylov subspaces.
- Large and sparse, nonsymmetric: GMRES with restarting, or one of the short-recurrence alternatives such as BiCGStab, chosen by experiment.
- Large sparse least squares: LSQR, which is conjugate gradients applied implicitly to the normal equations without ever forming them.
- A few eigenvalues of a large sparse matrix: Lanczos if symmetric, Arnoldi otherwise. These are the same Krylov subspaces again, asked a different question.
- Too large for any of the above, and only an approximate answer is wanted: sketch it. Chapter 24 replaces the matrix by a small random projection and accepts a result that is nearly optimal with high probability, which is the one escape from the cubic wall that this part does not otherwise offer.

In every case the first thing to establish is what κ is, because Theorem 17.6 bounds what any of them can deliver.

Insight

Direct and iterative methods differ in what they treat as the matrix.
Elimination needs the entries. It writes into them, it fills them in, and its cost is decided by the sparsity pattern, which is why Chapter 113 can predict that cost from a graph. An iterative method never sees an entry. It needs only the ability to compute Av for a given v, and so the matrix may be a function, a stencil, a convolution done by a fast transform, or a physical simulation with no matrix stored anywhere. That single change of interface is what makes solving a system with a billion unknowns possible, and it is a design decision rather than a mathematical one.
The mathematics then follows the interface. Once the only operation is $v \mapsto Av$, everything reachable in k steps lies in the Krylov subspace, and Chapter 44 shows that this is the same as saying the answer is a polynomial in A applied to b. So the convergence theory of every method in this chapter becomes a question about how small a polynomial of low degree can be made on the spectrum, and preconditioning becomes the art of changing the spectrum so that an easier polynomial suffices. The algorithms look different from one another; the analysis is one analysis.

Open Problem 18.11 (What actually governs the convergence of GMRES?)**.** *For a symmetric positive definite matrix, the convergence of the conjugate gradient method is completely understood: it depends on the eigenvalue distribution, and the classical bound in terms of the condition number is both provable and predictive. For a general nonsymmetric matrix, GMRES has no such theory.* What quantity predicts the convergence of GMRES?

What is known. *Not the eigenvalues. Greenbaum, Pták and Strakoš proved something remarkable and discouraging:* any *nonincreasing convergence curve can be produced by GMRES on a matrix with* any *prescribed spectrum [274]. So a matrix with tightly clustered eigenvalues can make GMRES stagnate for $n-1$ steps and then converge instantly, and a matrix with wildly spread eigenvalues can converge at once. The eigenvector conditioning is part of the story, and the pseudospectra of Chapter 72 are part of it, but no quantity is known that is both computable and predictive.*

Why this matters more than it sounds. *Every large-scale simulation that is not symmetric runs GMRES or a relative, and every preconditioner is designed against some heuristic about what will help. Without a theory, preconditioning is an experimental science. The known bounds, in terms of the field of values of Chapter 88 or the Crouzeix constant, apply only when the matrix is not too far from normal, which is exactly when the problem was not hard.*

Where a reader might start. *Construct the Greenbaum, Pták and Strakoš example at $n = 4$: choose the four eigenvalues you like, choose a convergence curve you like, and build the matrix. The construction is short and entirely constructive, and doing it once destroys the intuition that eigenvalues govern iterative convergence, which is the necessary first step towards asking what does.*

Exercises

Exercise 18.12. *For $A = \left(\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right)$ and $b = (1, 1)$, carry out three Jacobi steps and three Gauss–Seidel steps from $x_0 = (0, 0)$. Compute $\rho(G)$ for each and check that the observed decrease matches.*

Exercise 18.13. *Call A strictly diagonally dominant if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for every i. Show that Jacobi converges for such a matrix.* Hint: bound the entries of $G = \mathbf{1} - D^{-1}A$ row by row and use the fact that $\rho(G) \leq \max_i \sum_j |g_{ij}|$.

Exercise 18.14. *For $A = \left(\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right)$ and $v = (1, 0)$, compute K_1, K_2 and K_3. Why does the sequence stop growing, and what does that say about how many conjugate gradient steps are needed?*

Exercise 18.15. *Run two conjugate gradient steps on the matrix of Exercise 18.12 with $b = (1, 0)$, from $x_0 = (0, 0)$, and verify that x_2 is exact. Check also that $\langle Ap_0, p_1 \rangle = 0$, which is the property the method is named for. Then explain why the original right-hand side $b = (1, 1)$ finishes in a* single *step, so that p_1 never exists at all.*

Exercise 18.16. *A system has $\kappa = 10^4$. Using Theorem 18.7, roughly how many conjugate gradient iterations are needed to reduce the error by 10^{-6}? How many Jacobi iterations would be needed if $\rho(G) = 1 - 2/\kappa$? Compare.*

Exercise 18.17. *Let A be $n \times n$ with nonzeros only on the diagonal, in the first row and in the first column, an "arrowhead" matrix. Count its nonzeros, then count the nonzeros of its LU factors. Now reverse the ordering of the rows and columns and count again. What does the comparison show about the importance of ordering?*

Exercise 18.18. *Suppose an unpreconditioned solve takes 400 iterations at 1 unit of work each, and a preconditioner reduces this to 30 iterations but costs 6 units per iteration to apply on top of the matrix-vector product. Is it worth it? At what per-iteration cost does it stop being worth it?*

Exercise 18.19. *Verify directly that the vector with entries $v_i^{(j)} = \sin(ij\pi/(n+1))$ is an eigenvector of the tridiagonal matrix of Example 18.3 with eigenvalue $2 - 2\cos(j\pi/(n+1))$.* Hint: use the identity $\sin(\alpha - \beta) + \sin(\alpha + \beta) = 2 \sin \alpha \cos \beta$.

Exercise 18.20. *Let A be the $n^2 \times n^2$ matrix of the five-point Laplacian on an $n \times n$ grid: each row has a 4 on the diagonal and -1 for each of the up-to-four grid neighbours. How many operations does one product Ax cost, and how much storage does A need if the entries are never stored at all? Compare with $\tfrac{2}{3}(n^2)^3$ for a factorisation at $n = 1000$.*

References and further reading

Saad [538] is the standard reference on iterative methods for sparse systems; it covers everything in this chapter in proper detail, including the Chebyshev argument behind Theorem 18.7 and the many nonsymmetric alternatives we have only named. Trefethen and Bau [607] devote their final lectures to Krylov methods and are the best short account of why the polynomial view is the right one. Greenbaum's monograph is the place to go for what happens to conjugate gradients in floating point, which is subtler than this chapter has admitted: the exact-arithmetic termination after n steps does not survive rounding, and the method's practical behaviour was not properly understood until the 1990s. For preconditioning there is no single reference because the subject is as wide as the applications; Saad [538] is the starting point, and multigrid has a literature of its own. Golub and Van Loan [263] remains the encyclopaedia for the whole of Part VII.